



Hierarchical Ensemble-Based Feature Selection for Time Series Prediction

Huseyin Karaca, Aysin Tumay, Ali Taha Koc, Suleyman Serdar Kozat

GRC 2026
Bilkent University, Ankara, Türkiye

20 January 2026

Challenge: Time series prediction with high-dimensional features

Key Issues:

- **Curse of Dimensionality:** Limited samples vs. large feature count leads to overfitting
- **Feature Dominance:** Models overfit to dominant features (e.g., target lags), underutilizing side information

Limitation of Traditional Methods

Filter, wrapper, and embedded methods typically select a *single* dominant subset, leaving potentially valuable non-dominant features unused.

- **Filter Methods:** Statistical scoring (correlation, mutual information)
 - + Fast, model-agnostic
 - Univariate, ignore feature interactions
- **Wrapper Methods:** Forward/backward selection based on validation loss
 - + Multivariate, capture dependencies
 - Computationally expensive ($O(2^M)$ subsets)
- **Embedded Methods:** Integrated selection (Decision Trees, Random Forest)
 - + Efficient, simultaneous training
 - Not available for all models
- **Ensemble Methods:** Combine multiple models
 - + Robust predictions
 - Limited cross-group interaction exploitation

Hierarchical Ensemble-Based Feature Selection (HEFS)

Core Idea:

- Process features in **hierarchical layers**: Dominant $\rightarrow$ Non-Dominant
- Bridge layers by directly minimizing final loss by a cost optimization block
- Exploit both dominant and non-dominant groups

Key Innovation

Learnable scaling factor α_t enables interaction between feature groups across layers

Flexibility

Handles non-differentiable losses (L1, etc.) beyond standard Gradient Boosting

1 Hierarchical Utilization:

- Flexible framework for both dominant and non-dominant features
- Layered approach prevents information loss

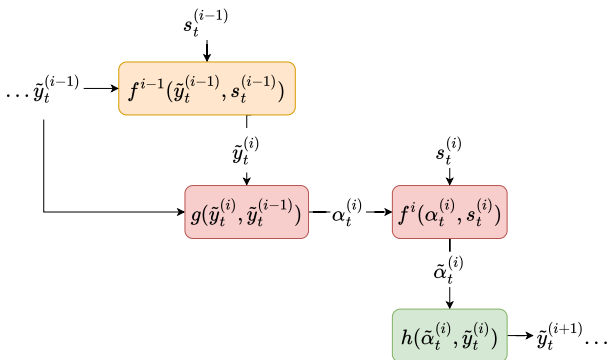
2 Beyond Twice-Differentiable Losses:

- Cost-optimization block adaptively refines outputs
- Circumvents gradient boosting limitations (e.g., L1 loss)

3 Multivariate Cross-Group Modeling:

- Captures dependencies across feature groups
- Layer-wise optimization enables complex interactions

4 Open-Source: Implementation publicly available



- **Layer 1 (Orange):** Train on dominant features $s_t^{(1)}$ (e.g., target lags)
- **Cost Optimization (Red):** Find optimal scaling $\alpha_t^{(i)}$
- **Layer 2 (Red):** Predict $\alpha_t^{(i)}$ using non-dominant features $s_t^{(2)}$
- **Final Output (Green):** $\tilde{y}_t^{(i+1)} = (1 + \tilde{\alpha}_t^{(i)})\tilde{y}_t^{(i)}$

Challenges:

- Performance of gradient boosting may degrade with non-differentiable losses
- No direct interaction mechanism between hierarchical feature groups

Our Solution: Introduce sample-wise scaling factor $\alpha_t^{(i)}$

$$\hat{y}_t^{(i)} = \alpha_t^{(i)} \tilde{y}_t^{(i)}$$

Optimization Objective:

$$\alpha_t^{(i)} = \underset{\alpha \in [1-\beta, 1+\beta]}{\operatorname{argmin}} \ell(y_t, \alpha \tilde{y}_t^{(i)})$$

Advantages

- **No differentiability required:** Works with any loss function
- **Enables cross-group interaction:** Bridges predictions from different feature groups

HEFS Algorithm (K=2):

- 1** Initialize: Assign dominant features to $\mathbf{s}_t^{(1)}$
- 2** Train first model: $\tilde{y}_t^{(1)} = f^{(1)}(\mathbf{s}_t^{(1)}, y_t)$
- 3** **Cost Optimization:**
 - For each sample t , find $\alpha_t^{(2)}$ that minimizes $\ell(y_t, \alpha \tilde{y}_t^{(1)})$
- 4** Train second model: $\tilde{\alpha}_t^{(2)} = f^{(2)}(\mathbf{s}_t^{(2)}, \tilde{y}_t^{(1)}, \alpha_t^{(2)})$
- 5** Final prediction: $\tilde{y}_t^{(3)} = (1 + \tilde{\alpha}_t^{(2)})\tilde{y}_t^{(1)}$

Note

Can extend to $K > 2$ layers for deeper hierarchies

Datasets:

- **Synthetic:** ARMA(4,5) process with dominant and non-dominant 36 features
- **M4 Yearly:** Competition dataset, financial/economic series
- **Weather:** Temperature prediction, meteorological variables

Baseline Methods:

- | | |
|----------------------|-----------------------|
| ■ Filter (Pearson) | ■ Embedded (LightGBM) |
| ■ Wrapper (Forward) | ■ Standard Ensemble |
| ■ Wrapper (Backward) | ■ Baseline LightGBM |

Evaluation:

- 3-fold TimeSeriesSplit cross-validation
- 30 repeated trials with different random seeds
- Paired one-sided t -test for significance ($p < 0.05$)

Table: Stationarity analysis and dimensionality of the datasets.

Dataset	Num. features	Num. samples	Combined KPSS p -values
Synthetic	36	300	5.4×10^{-11}
M4 Yearly	33	400	4.4×10^{-3}
Weather	40	300	1.0×10^{-2}

Feature Engineering:

- **Dominant:** Lagged values, rolling statistics
- **Non-dominant:** Calendar features (M4), side info (Weather)

Test Performance (MSE)



Table: Test MSE (lower is better).

Model	Synthetic	M4	Weather
Filter	1.023	0.105	0.0123
Forward Wrapper	1.004	0.106	0.0111
Backward Wrapper	0.980	0.107	0.0110
Embedded	0.978	0.109	0.0135
Ensemble	0.965	0.111	0.0134
Baseline LightGBM	0.980	0.105	0.0120
HEFS ($K=2$)	0.956	0.103	0.0110
HEFS ($K=3$)	0.977	0.111	0.0153
HEFS (Ablation)	0.962	0.106	0.0122

- HEFS achieves **lowest MSE** on all datasets
- Cost optimization block is crucial for performance

Training Performance (MSE)



Table: Train MSE (lower is better).

Model	Synthetic	M4	Weather
Filter	0.900	0.0739	0.00382
Forward Wrapper	0.697	0.0379	0.00168
Backward Wrapper	0.858	0.0746	0.00214
Embedded	0.961	0.0210	0.00721
Ensemble	0.655	0.0281	0.00530
Baseline LightGBM	0.941	0.0388	0.00447
HEFS ($K=2$)	0.725	0.0235	0.00630
HEFS ($K=3$)	0.644	0.0673	0.00451
HEFS (Ablation)	0.468	0.0202	0.00570

Key Observation:

- HEFS shows **smaller train-test gap** $\Rightarrow$ better generalization
- Cost optimization block reduces overfitting

Table: Training times (in seconds).

Model	Synthetic	M4	Weather
Filter	0.316	1.763	6.048
Forward Wrapper	153.2	71.38	128.9
Backward Wrapper	265.9	135.5	99.60
Embedded	0.531	11.89	2.572
Ensemble	11.74	13.26	3.533
Baseline LightGBM	0.556	6.309	0.591
HEFS ($K=2$)	2.478	10.56	2.238
HEFS ($K=3$)	3.779	5.453	1.263
HEFS (Ablation)	4.231	11.16	1.407

Key Takeaway:

- HEFS is **10-20 $\times$ faster** than wrapper methods
- Comparable to ensemble methods with superior accuracy

Summary:

- Hierarchical ensemble that utilizes both dominant and non-dominant features
- Cost Optimization Block enables layer interaction without differentiability constraints
- Better accuracy and robustness on synthetic and real-world data
- Faster than wrapper methods, practical for real use

Limitations:

- Manual feature grouping
- Benefits may diminish with abundant training data

Future Work:

- Automated feature grouping strategies
- Deeper hierarchies with weaker base learners

Questions?



Source Code

[github.com/huseyin-karaca/
hierarchical-feature-selection](https://github.com/huseyin-karaca/hierarchical-feature-selection)